Revision Games — 1.2 Software & Software Development

OCR A Level Computer Science (H446) — Component 01, Section 1.2 (1.2.1 Operating systems · 1.2.2 Applications generation · 1.2.3 Software development · 1.2.4 Types of programming language)

A pack of low-prep games for retrieval practice. Print, cut where indicated, and play.

Loop cards (dominoes)

How to play. Cut out the 15 cards and deal them out (one or more per student). One person starts by reading the "Who has…?" clue on their card. Whoever holds the matching **term** reads it aloud ("I have…") and then reads their own clue. Play continues until the loop returns to the start. The loop is **closed** — Card 15's clue leads back to Card 1.

1. **Card 1 — I have: Operating system.** Who has: the OS job of allocating RAM to processes, using techniques like paging and segmentation?
2. **Card 2 — I have: Memory management.** Who has: dividing memory into **fixed-size** blocks called pages and frames?
3. **Card 3 — I have: Paging.** Who has: dividing memory into **variable-size**, logical blocks?
4. **Card 4 — I have: Segmentation.** Who has: using secondary storage **as if it were RAM** to run more or larger programs?
5. **Card 5 — I have: Virtual memory.** Who has: the slowdown caused by **excessive** swapping of pages between RAM and disk?
6. **Card 6 — I have: Disk thrashing.** Who has: a signal that a device or process needs the **processor's attention**?
7. **Card 7 — I have: Interrupt.** Who has: the data structure (LIFO) onto which registers are pushed so a job can resume after an interrupt?
8. **Card 8 — I have: Stack.** Who has: the scheduling algorithm giving each process a fixed **time slice (quantum)** in turn?
9. **Card 9 — I have: Round robin.** Who has: the firmware that runs **first** at start-up, performs the POST, and loads the OS?
10. **Card 10 — I have: BIOS.** Who has: the translator that turns high-level source into machine code **all at once**, producing an executable?
11. **Card 11 — I have: Compiler.** Who has: the **first** stage of compilation, which breaks code into tokens and builds the symbol table?
12. **Card 12 — I have: Lexical analysis.** Who has: the development methodology built around **risk analysis** at every iteration?
13. **Card 13 — I have: Spiral.** Who has: the OOP feature where a subclass **acquires** the attributes and methods of a superclass?
14. **Card 14 — I have: Inheritance.** Who has: the addressing mode where the operand **is** the actual value to be used?
15. **Card 15 — I have: Immediate addressing.** Who has: the software that manages hardware and provides a platform for applications?

(Card 15 leads back to Card 1 — loop verified closed.)

Taboo cards

How to play. In pairs/teams. One student describes the **TERM** at the top of the card to their partner **without saying any of the 4 banned words** (or close variants). Score 1 point per term guessed in 60 seconds; pass if stuck.

1. INTERPRETER Banned: line · execute · run · slow

2. COMPILER Banned: executable · machine code · all · once

3. VIRTUAL MEMORY Banned: RAM · secondary storage · swap · disk

4. INTERRUPT Banned: signal · processor · attention · ISR

5. PAGING Banned: fixed · blocks · frames · memory

6. ENCAPSULATION Banned: data · hiding · private · methods

7. POLYMORPHISM Banned: method · different · object · override

8. AGILE Banned: iterative · feedback · change · increments

Bingo

How to play. Each player draws a 3×3 grid and fills it with any **9** terms from the word bank below. The caller reads clues from the numbered list in random order (tick them off). Players cross off the matching term. First to a line (or full house) calls "Bingo!" — verify against the clue list.

Word bank (20 terms):

Operating system · Paging · Segmentation · Virtual memory · Disk thrashing · Interrupt · ISR · Round robin · Real-time OS · BIOS · Device driver · Compiler · Interpreter · Assembler · Linker · Loader · Waterfall · Agile · Encapsulation · Polymorphism

Caller's clues:

1. Software that manages hardware and provides a platform for applications. → *Operating system*
2. Dividing memory into fixed-size blocks (pages and frames). → *Paging*
3. Dividing memory into variable-size, logical blocks. → *Segmentation*
4. Using secondary storage as if it were RAM. → *Virtual memory*
5. Excessive swapping between RAM and disk that slows the system. → *Disk thrashing*
6. A signal that a device or process needs the processor's attention. → *Interrupt*
7. The OS code run to deal with one particular interrupt. → *ISR*
8. Scheduling that gives each process a fixed time slice in turn. → *Round robin*
9. An OS that guarantees a response within a set time limit. → *Real-time OS*
10. Firmware that runs first, performs the POST and loads the OS. → *BIOS*
11. Software that lets the OS control a specific piece of hardware. → *Device driver*
12. Translates high-level source into machine code all at once. → *Compiler*

13. Translates and executes high-level source one line at a time. → *Interpreter*
14. Translates assembly language into machine code (≈ one-to-one). → *Assembler*
15. Combines a compiled program with libraries/modules into one executable. → *Linker*
16. OS software that copies an executable into RAM ready to run. → *Loader*
17. A linear methodology where one stage is completed before the next. → *Waterfall*
18. An iterative methodology with continuous client feedback. → *Agile*
19. Bundling data with methods and hiding the data. → *Encapsulation*
20. The same method name behaving differently per object. → *Polymorphism*

Quick-fire 'Guess the term'

How to play. Read a clue; students write or call the answer. Keep score; aim for speed. Answers are in brackets — cover them when reading.

1. The data structure (LIFO) used to save registers during an interrupt. → **(Stack)**
2. A page is needed but it is currently on disk, so the OS must swap it in. → **(Page fault)**
3. Pre-written, pre-compiled, tested subroutines you can reuse in your program. → **(Library)**
4. The second stage of compilation, where tokens are checked against the grammar. → **(Syntax analysis / parsing)**
5. A scheduling property: a running process can be stopped and the CPU reallocated. → **(Pre-emptive)**
6. Platform-independent code (e.g. Java bytecode) run by a virtual machine. → **(Intermediate code)**
7. A template/blueprint that defines a class's attributes and methods. → **(Class)**
8. A specific instance of a class. → **(Object)**
9. The addressing mode where the operand is the address holding the value. → **(Direct addressing)**
10. The addressing mode where the operand points to a location holding the address of the value. → **(Indirect addressing)**
11. The XP practice of two programmers working at one machine. → **(Pair programming)**
12. The methodology built around rapid prototyping refined by user feedback. → **(RAD)**